

Simulador de Parqueadero Inteligente con Hilos y GUI en Python

Estudiante:	Juan Manuel Pedraza Sánchez
Programa:	Ingeniería de Sistemas
Docente:	Diana Carolina Beltran Peña
Fecha:	Mayo de 2026

1. Descripción del Sistema y Analogía con Sistemas Operativos

El simulador de parqueadero inteligente modela el comportamiento de un sistema operativo (SO) en la gestión de recursos limitados y procesos concurrentes. En este proyecto, el parqueadero actúa como el SO, los espacios de parqueo representan los recursos del sistema (analogías a la CPU o la memoria RAM), y cada vehículo es un proceso independiente que compete por acceder a dichos recursos.

La siguiente tabla resume la analogía directa entre el simulador y un Sistema Operativo real:

Simulador de Parqueadero	Sistema Operativo
Vehículo	Proceso
Espacio de parqueo	CPU / Memoria
Semáforo (capacidad)	Planificador de recursos
Lock / Mutex	Exclusión mutua en sección crítica
Vehículo esperando	Proceso bloqueado en cola de listos
Vehículo estacionado	Proceso en ejecución
Vehículo saliendo	Proceso terminado

Tabla 1. Analogía entre el simulador y un Sistema Operativo.

El simulador implementa el ciclo de vida completo de un proceso: el vehículo llega (estado Nuevo), espera disponibilidad (estado Listo/Bloqueado), ingresa al parqueadero (estado En Ejecución) y finalmente sale (estado Terminado). Este ciclo se ejecuta de forma concurrente para múltiples vehículos simultáneos, reproduciendo fielmente el comportamiento multitarea de un SO moderno.

2. Uso de Hilos, Sincronización y Gestión de Recursos

2.1 Hilos (Threads)

Cada vehículo se implementa como un hilo independiente mediante la clase `threading.Thread` de Python. Todos los hilos comparten el mismo espacio de memoria del proceso principal, lo que permite el acceso concurrente a variables globales como el contador de espacios ocupados. La creación y lanzamiento de cada hilo-vehículo se realiza de la siguiente manera:

```
t = threading.Thread(target=vehiculo, args=(nombre,), daemon=True)
t.start()
```

2.2 Semáforo (Semaphore)

El semáforo controla el acceso al recurso limitado (espacios de parqueo). Se inicializa con el valor de la capacidad máxima del parqueadero. Cuando un vehículo intenta ingresar, llama a `semaforo.acquire()`: si hay cupos disponibles el contador decrece y el hilo continúa, pero si el parqueadero está lleno, el hilo queda bloqueado hasta que otro vehículo libere un espacio con `semaforo.release()`. Este mecanismo replica exactamente como un planificador de SO bloquea y desbloquea procesos según la disponibilidad de recursos.

```
semaforo = threading.Semaphore(CAPACIDAD) # Ej: CAPACIDAD = 5
semaforo.acquire() # Bloquea si no hay cupo
semaforo.release() # Libera al salir
```

2.3 Lock y Exclusión Mutua

El `threading.Lock()` protege la sección crítica donde se modifica el contador de espacios ocupados. Sin este mecanismo, dos hilos podrían leer y escribir simultáneamente la misma variable, generando una condición de carrera (race condition) que produciría valores incorrectos. El uso de `with lock:` garantiza que solo un hilo a la vez pueda modificar el estado compartido, asegurando la consistencia de los datos en todo momento.

2.4 Comunicación Hilos - GUI con Queue

Tkinter solo permite actualizaciones de interfaz desde el hilo principal. Para comunicar los hilos-vehículo con la GUI sin bloqueos ni condiciones de carrera, se utiliza `queue.Queue()`: los hilos depositan eventos (llegada, entrada, salida) en la cola, y el hilo principal los consume cada 200 ms mediante `root.after(200, leer_eventos)`, actualizando los elementos visuales de forma segura y sin interrumpir la ejecución de los hilos.

2.5 Métricas y Registro de Eventos

El módulo “`metricas.py`” registra los tiempos de llegada, entrada y salida de cada vehículo usando `time.time()`. Con estos datos calcula el tiempo promedio de espera (tiempo entre llegada e ingreso) y el tiempo promedio de estancia. Todos los eventos se exportan automáticamente a un archivo `logs/eventos.csv` que permite el análisis posterior del comportamiento del sistema bajo diferentes cargas de vehículos.

Conclusiones

El simulador de parqueadero inteligente logra representar de forma práctica y visual los conceptos fundamentales de los Sistemas Operativos: concurrencia mediante hilos, sincronización con semáforos y locks, gestión de recursos limitados y el ciclo de vida de los procesos. La interfaz gráfica desarrollada en Tkinter permite observar en tiempo real como el sistema planifica el acceso a recursos, replica el comportamiento de bloqueo y desbloqueo de procesos, y evidencia visualmente las consecuencias de la exclusión mutua y la condición de carrera cuando no se aplica correctamente la sincronización.